Propósito y alcance del sistema

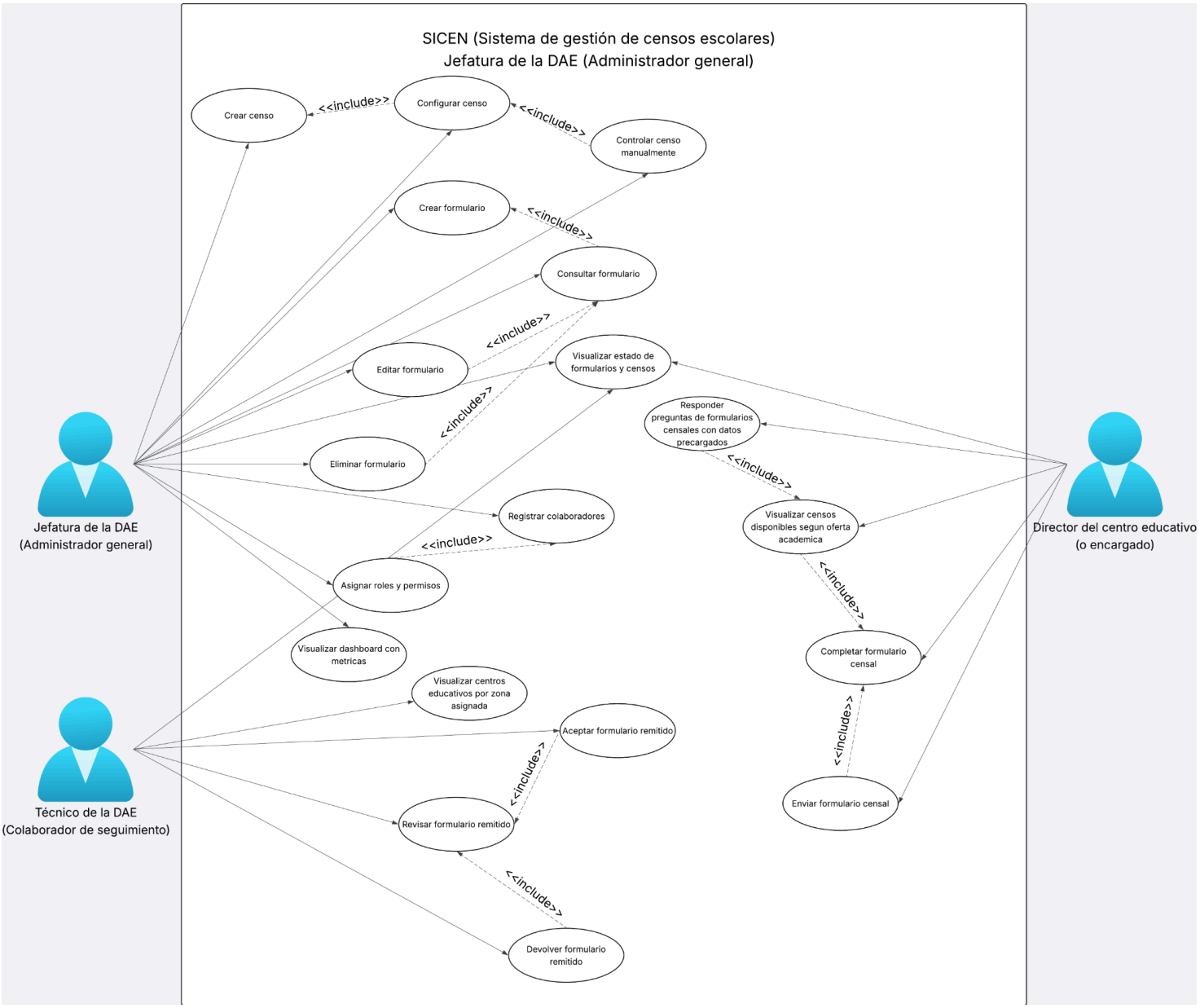
El sistema SICEN propone organizar, reunir y gestionar los procesos de levantamiento, registro, seguimiento y validación de información estadística de los centros educativos públicos y privados del país, sustituyendo los registros manuales y el intercambio de información por medios dispersos que actualmente utiliza el Ministerio de Educación Pública (MEP). El sistema permitirá:

- Construir, actualizar y eliminar formularios censales personalizados (desde cero o mediante plantillas), definiendo el tipo de respuesta de cada pregunta.
- Configurar y dar seguimiento a censos, estableciendo fechas de apertura/cierre y los formularios que los componen según la oferta educativa de cada centro.
- Registrar colaboradores de la DAE y asignarles roles (administrador, técnico, director), controlando así el acceso a las funcionalidades del sistema.
- Completar, remitir y revisar formularios censales conforme a un flujo de estados definido (borrador, en registro, en revisión, en subsane, aceptado).
- Consultar visores y reportes con métricas del avance de los censos vigentes.
- Autenticar a la Jefatura de la DAE mediante su correo institucional (@mep.go.cr), sin necesidad de contraseña.

Al desarrollarse como un trabajo individual, el alcance del sistema se limita a los perfiles Jefatura de la DAE, Técnico de la DAE y Director del centro educativo, excluyendo el perfil Supervisor y algunas funcionalidades originalmente contempladas para los demás roles. En esta fase, el sistema se plantea como una propuesta de solución tecnológica que, en un escenario real, podría integrarse como un nuevo módulo de la plataforma ministerial.

Modelado del sistema

Diagrama de casos de uso



Interfaz de usuario (UI/UX)

Se creó un wireframe de fidelidad media haciendo uso de la herramienta Figma, este prototipo viene a funcionar como una propuesta estructural para el sistema de SICEN, por lo que se toman en cuenta las siguientes consideraciones sobre el mismo:

- Layout y estructura definidos: columnas, espaciados, tamaños relativos.

- Tipografía genérica pero legible, sin la fuente final de marca.
- Paleta de colores limitada o en escala de grises, sin la paleta final de marca.
- Contenido real de ejemplo (textos representativos, no lorem ipsum si se puede evitar).
- Componentes básicos: botones, tarjetas, inputs, con estilos simples pero reconocibles.
- Interacciones simuladas básicas: navegación entre pantallas, clics en botones, hover sencillos.
- Estados principales de algunos elementos (activo, hover, deshabilitado) de forma esquemática.

Enlace al wireframe en Figma:
https://www.figma.com/design/OGuru9EdHMBLaYH5Wwfplk/wireframe-sicen?node-id=0-1&t=dkofCKivdvBxqkrt-1

Adicionalmente, se subirá una carpeta llamada “prototipo” a la carpeta “documentación” del branch “docs” del repositorio de GitHub, en la cual se encontrarán individualmente colocadas todas las pantallas del wireframe con su respectivo nombre.

Guía de estilos

Al trabajar en el diseño del sistema se hará uso de los lineamientos establecidos por el Ministerio de Educación Pública (MEP) en su **Manual de identidad gráfica 2026 - 2030**. De acuerdo a las pautas establecidas en dicha documentación, el sistema denotará los siguientes estilos:

Paleta de colores

Colores Primarios

- **Dorado Institucional / Ocre** (R:207 G:172 B:101, HEX: #cfac65)

- **Azul Marino Oscuro / Sombra** (R:2 G:26 B:83, HEX: #021a53)
- **Azul Real / Institucional** (R:0 G:46 B:122, HEX: #002e7a)

Colores Secundarios

- **Gris Claro / Plata** (R:193 G:197 B:200, HEX: #c1c5c8)
- **Azul Francia / Brillant** (R:22 G:70 B:135, HEX: #164687)
- **Blanco** (R:255 G:255 B:255, HEX: #ffffff)

Color Auxiliar

- **Gris Pizarra** (R:92 G:102 B:112, HEX: #5c6670): Color auxiliar para cumplir el contraste mínimo de 4.5:1 en texto secundario y en el badge Cerrado.
- **Dorado Oscuro / Bronce** (R:138 G:106 B:31, HEX: #8a6a1f): Color auxiliar derivado del Dorado Institucional, utilizado en los íconos decorativos de las tarjetas de métrica en modo claro, ya que el Dorado Institucional no se distingue sobre su fondo.

Tipografía

Se utilizará la tipografía Baskerville a lo largo del sistema. Para su implementación web se utiliza Libre Baskerville, versión de Baskerville optimizada para pantalla y de licencia abierta, con Baskerville como fuente de respaldo.

Componentes

Componente	Uso en SICEN	Estilo con identidad institucional
Logo / marca	Encabezado de login y navbar de todas las vistas autenticadas	Ícono en Azul Marino Oscuro, “SICEN” en Libre Baskerville bold

Tarjeta de selección de rol	Pantalla de login. Selección de tipo de usuario	Fondo blanco, borde Gris Claro. Ícono circular con fondo Azul Francia claro
Botón primario	“Continuar al sistema del MEP”, “Iniciar sesión”, “Remitir formulario”, “+ Nuevo Censo”, “Aceptar formulario”, “Guardar Cambios”	Fondo Azul Real, texto blanco, Libre Baskerville bold
Botón secundario (outline)	“Continuar” (tarjetas de selección de tipo de usuario), “Volver a la selección de usuario”, “Guardar borrador”, “Cancelar”	Fondo transparente, borde y texto Azul Real
Botón de alerta/corrección	"Devolver para corrección"	Fondo Dorado Institucional, texto Azul Marino Oscuro, para diferenciarlo visualmente de una acción positiva
Campo de texto (input)	Correo electrónico, contraseña, nombre de formulario/censo, buscador	Borde Gris Claro, texto Libre Baskerville; borde activo en Azul Francia al enfocar
Barra lateral (sidebar)	Navegación del Panel de Administración (Dashboard, Formularios, Censos, Colaboradores) y de la vista del Técnico	Fondo Azul Marino Oscuro, texto blanco; ítem activo con fondo Azul Real y franja izquierda en Dorado Institucional
Encabezado superior (navbar)	Barra con usuario/avatar en todas las vistas autenticadas	Fondo blanco, borde inferior Gris Claro; avatar circular con iniciales sobre fondo

		Azul Francia
Tarjeta de métrica (stat card)	“Censos Activos”, “Formularios Creados”, “Centros Asignados”, “Respuestas Recibidas”	Fondo blanco, borde Gris Claro; cifra destacada en Azul Marino Oscuro; ícono decorativo en Dorado Oscuro sobre fondo Dorado Institucional al 18 % (en modo oscuro, ícono en Dorado Institucional).
Tabla de datos	“Censos Vigentes”, “Formularios en Revisión”	Encabezado en Azul Marino Oscuro con texto blanco; filas alternadas blanco / Gris Claro al 10%
Etiqueta de estado (badge)	Abierto / Pausado / Cerrado; Borrador / En registro / En revisión / En subsane / Aceptado	Borrador: fondo Gris Claro, texto Azul Marino Oscuro. En registro: fondo blanco, borde y texto Azul Francia. En revisión: fondo Azul Real, texto blanco. Abierto y Aceptado: fondo Azul Francia, texto blanco. Pausado y En subsane: fondo Dorado Institucional, texto Azul Marino Oscuro. Cerrado: fondo Gris Pizarra, texto blanco
Tarjeta de pregunta	“Preguntas Agregadas” en el Constructor de Formularios	Fondo blanco, borde Gris Claro; tipo de pregunta (Selección Única, etc.) como badge con fondo Azul Francia y texto blanco
Menú desplegable (select)	“Tipo de Respuesta”,	Mismo estilo que input, con

	selector de Centro Educativo, selector de Estado	flecha indicadora en Azul Marino Oscuro
Checkbox	“Niveles educativos que se imparten”	Sin marcar: fondo blanco, borde Gris Claro. Marcado: relleno Azul Francia con marca blanca
Radio button	“¿El centro cuenta con acceso a internet?”	Punto activo en Azul Francia
Área de texto (textarea)	“Observaciones”, “Motivo de devolución”	Mismo estilo que input, con mayor altura
Barra de progreso	“Progreso general” en la vista del Director	Track en Gris Claro, relleno en Azul Francia
Barra de búsqueda	“Buscar censo...”, búsqueda rápida en vista Técnico	Igual que input, con ícono decorativo de lupa en Gris Claro
Texto secundario	Fechas de vencimiento, textos de ayuda, subtítulos	Gris Pizarra
Interruptor de modo claro/oscuro	Esquina superior de las pantallas de acceso y encabezado de las vistas autenticadas (RNF-001)	Botón con ícono de sol/luna, texto visible ‘Modo oscuro’ y aria-pressed; mismo estilo que el botón secundario
Modo oscuro	Todas las vistas (RNF-001)	Fondo general Azul Marino Oscuro; tarjetas y superficies en Azul Real; texto blanco; texto secundario y bordes en Gris Claro; acentos y foco en Dorado Institucional; botón primario con fondo Dorado y

		texto Azul Marino Oscuro; botón de corrección con fondo transparente, borde y texto Dorado, para diferenciarlo del botón primario; checkbox y radio button marcados con relleno Dorado y marca o punto en Azul Marino Oscuro.

Restricciones de usabilidad y accesibilidad (Requerimientos no funcionales de accesibilidad y usabilidad)

RNF-001: Modo oscuro

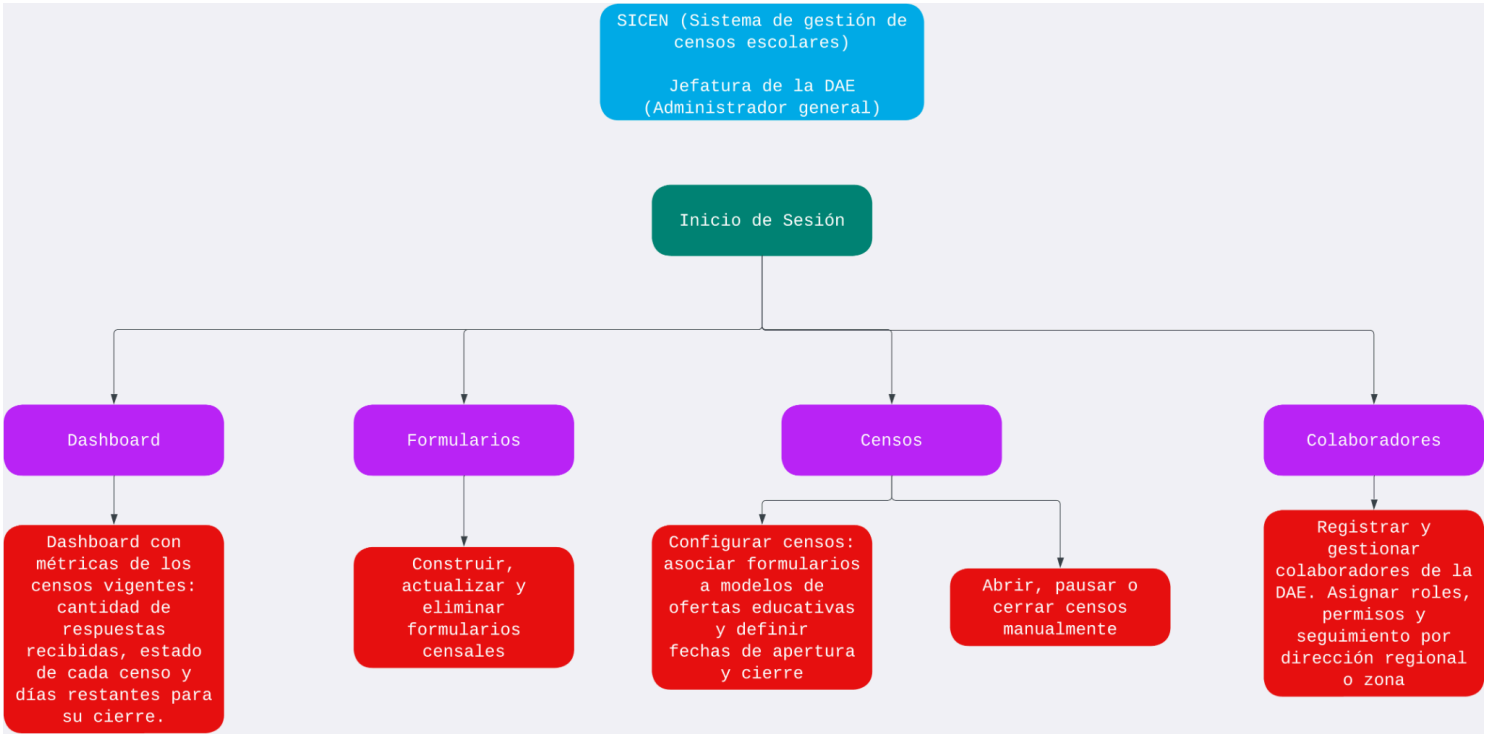
El sistema debe ofrecer un modo oscuro, el cual contará con una detección automática en caso de que el usuario utilice un tema oscuro en su equipo, esto con el fin de facilitar su uso a personas con baja visión. Adicionalmente, este modo también será activable y desactivable desde cualquier pantalla.

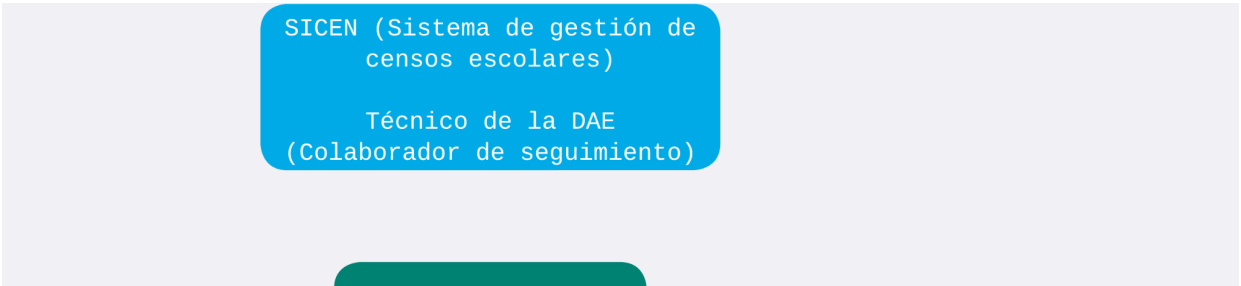
RNF-002: Cumplimiento de pautas de accesibilidad web

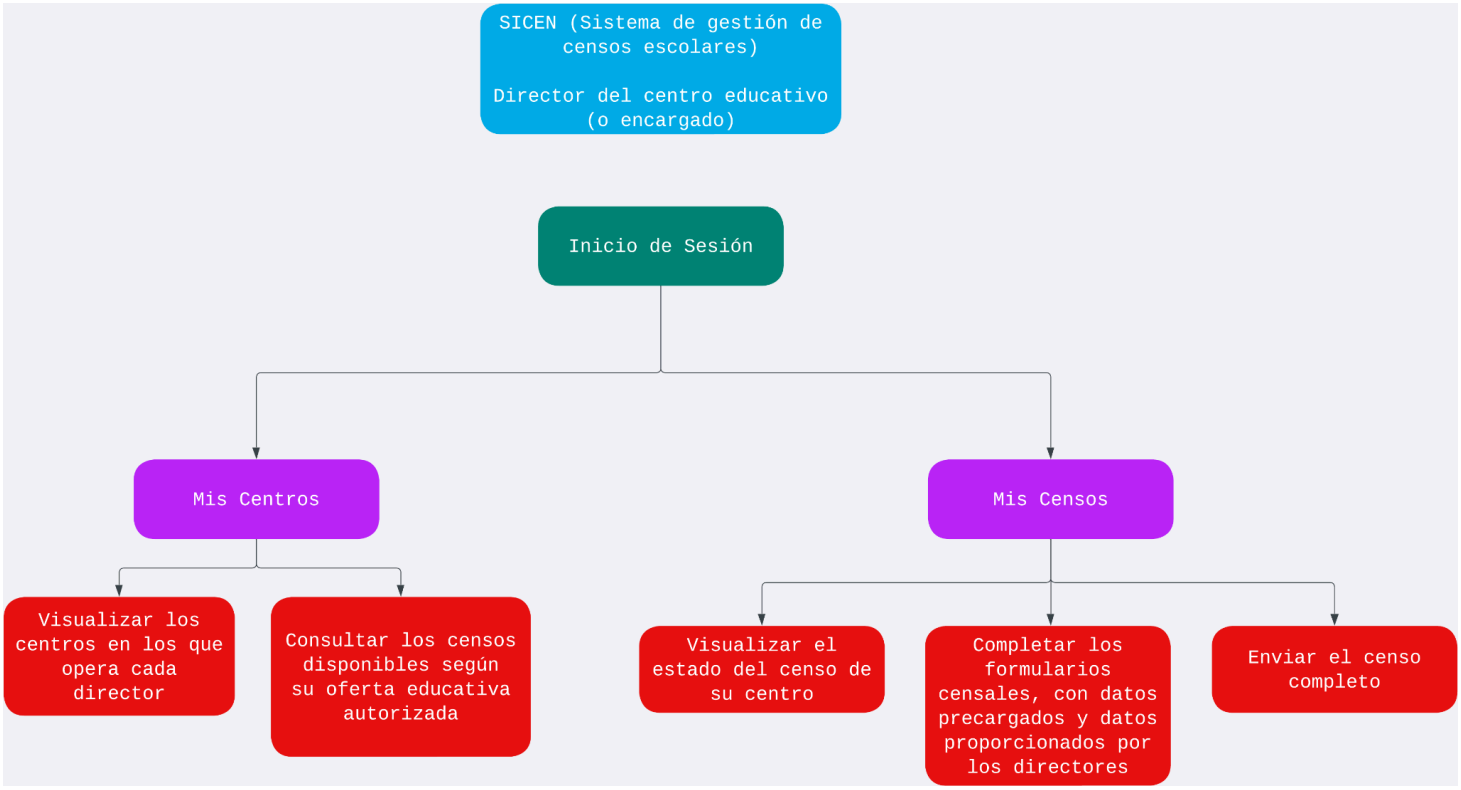
El sistema debe cumplir con las pautas de accesibilidad web (WCAG), de manera que, al evaluar cada pantalla con la herramienta de pruebas automáticas WAVE (extensión de Google Chrome), no se reporte ningún error de accesibilidad.

Flujo de navegación entre pantallas (mapa de sitio)

Diseño de navegación

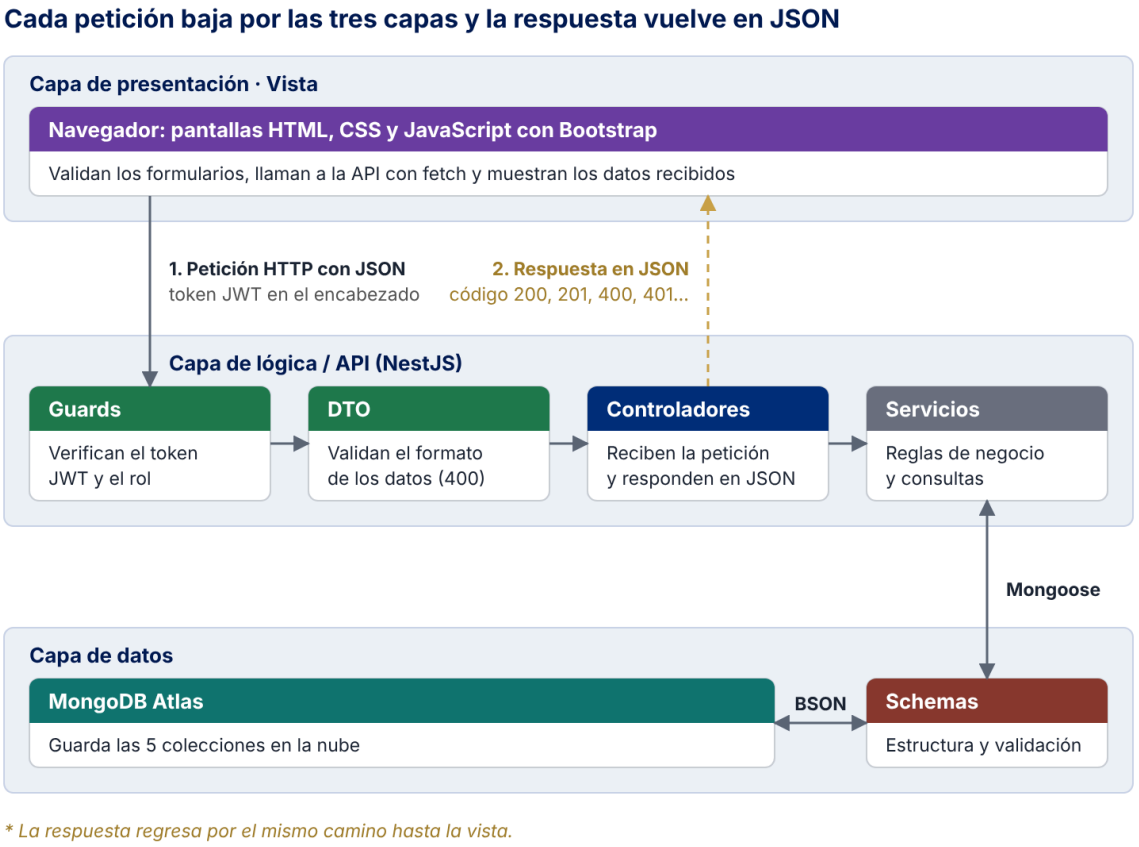





SICEN (Sistema de gestión de censos escolares)
Director del centro educativo (o encargado)
Inicio de Sesión
Mis Centros
Mis Censos
Visualizar los centros en los que opera cada director
Consultar los censos disponibles según su oferta educativa autorizada
Visualizar el estado del censo de su centro
Completar los formularios censales, con datos precargados y datos proporcionados por los directores
Enviar el censo completo

Arquitectura del sistema

SICEN usa una arquitectura cliente-servidor de tres capas (presentación, lógica/API y datos), organizada con el patrón Modelo-Vista-Controlador (MVC). El back-end se construye con NestJS como API REST.



** La respuesta regresa por el mismo camino hasta la vista.*

La petición baja desde la vista y pasa por guards, DTO, controladores, servicios y schemas hasta MongoDB Atlas; la respuesta regresa por el mismo camino y llega a la vista en JSON.

Capas

Capa	Componentes	Componente MVC	Responsabilidad
Presentación	Pantallas HTML, CSS y JavaScript con Bootstrap, en el navegador	Vista	Muestra las pantallas, valida los formularios antes de enviarlos y consume la API con fetch
Lógica / API	Controladores, guards y DTO de NestJS	Controlador	Recibe las peticiones HTTP, verifica el token y el rol (guards), valida el formato de los datos (DTO) y responde en JSON
Lógica / API	Servicios de NestJS	Modelo (lógica)	Aplica las reglas de negocio y consulta la base de datos
Datos	Schemas de Mongoose y MongoDB Atlas	Modelo (datos)	Define la estructura y las validaciones de las colecciones y guarda los documentos

Comunicación entre capas

Origen → destino	Mecanismo	Formato
Vista → API	Petición HTTP (GET, POST, PUT, PATCH, DELETE) a la ruta del recurso (por ejemplo /censos o /formularios) con fetch; el token JWT viaja en el encabezado Authorization: Bearer <token>	JSON
API → Vista	Respuesta HTTP con código de estado (200, 201, 400, 401, 403, 404, 409)	JSON
Controlador → Servicio	El controlador llama a los métodos del servicio, que NestJS le entrega por inyección de dependencias	Objetos JavaScript (DTO)
Servicio → Schema	Métodos del modelo de Mongoose inyectado con @InjectModel (find, create, findByIdAndUpdate, etc.)	Objetos JavaScript
Schema → MongoDB Atlas	Driver de MongoDB por conexión cifrada (TLS); la cadena de conexión se guarda en una variable de entorno	BSON

Ejemplo del recorrido de una petición (RF-009, el director remite un formulario):

1. La vista envía PATCH /respuestas-censo/:id/enviar con las respuestas en JSON y el token del director.
2. Los guards verifican el token y que el rol sea director; si no, responden 401 o 403.
3. ValidationPipe revisa el cuerpo contra su DTO; si el formato es incorrecto, responde 400.
4. El controlador llama al método enviar del servicio de respuestas de censo.
5. El servicio verifica que el censo esté abierto y que las preguntas obligatorias tengan respuesta, y cambia el estado a en_revision; si una regla no se cumple, responde 409.
6. El schema valida el documento y Mongoose lo guarda en MongoDB Atlas.
7. La API responde 200 con el documento actualizado y la vista actualiza el estado en pantalla.

Patrón arquitectónico

Modelo-Vista-Controlador (MVC): patrón que organiza el desarrollo dividiendo el sistema en tres componentes: modelo, vista y controlador. El modelo gestiona los datos y la lógica, la vista se encarga de la presentación y el controlador funciona como intermediario entre ambos.

Aplicación en SICEN: la vista son las pantallas del front-end (HTML, CSS y JavaScript con Bootstrap). Los controladores de NestJS reciben las peticiones de la vista y devuelven respuestas en JSON. El modelo lo forman los servicios, que contienen la lógica de negocio y el acceso a los datos, y los schemas de Mongoose, que definen la estructura y las validaciones de las colecciones almacenadas en MongoDB Atlas.

Justificación: separar estas responsabilidades permite modificar la interfaz sin afectar el acceso a los datos, y viceversa; además, facilita ubicar cada parte del código y probar cada componente por separado.

No se emplean patrones adicionales. Los guards, los DTO y la inyección de dependencias son mecanismos propios de NestJS, y los servicios forman parte del modelo según la definición de MVC (datos y lógica).

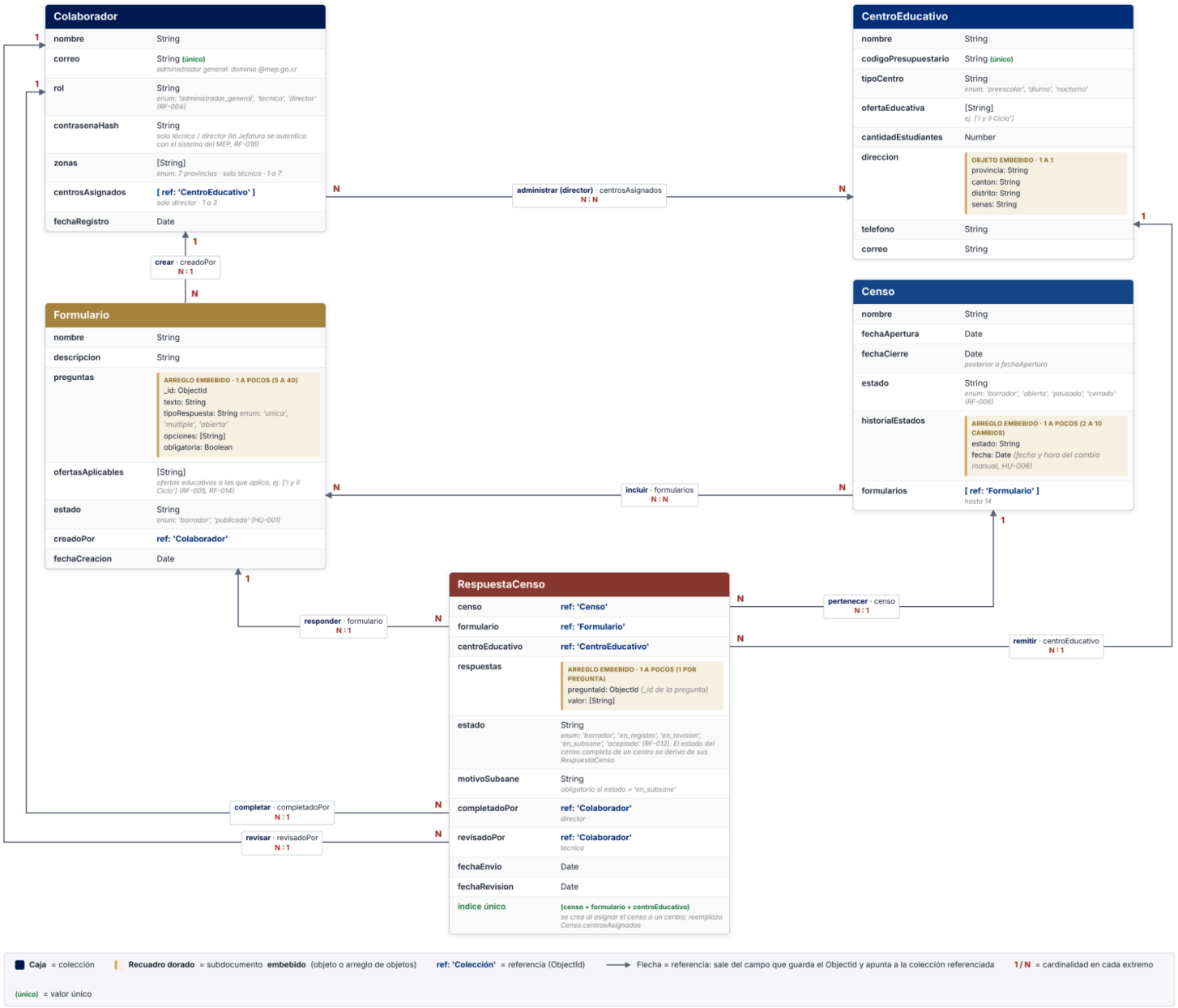
Diseño de la base de datos

La base de datos tiene 5 colecciones en MongoDB Atlas: Colaborador, CentroEducativo, Formulario, Censo y RespuestaCenso.

El diagrama de colecciones se muestra en la siguiente página: cada caja es una colección, los recuadros dorados son subdocumentos embebidos y las flechas salen del campo que guarda la referencia. (El diagrama también puede ser encontrado en el repositorio del proyecto, en la rama docs/, dentro de la carpeta documentación bajo el nombre modeloBD)

SICEN — Modelado de colecciones (MongoDB)

Sistema de Gestión de Censos Escolares del MEP: configura censos escolares, los asigna a los centros educativos y recopila las respuestas de sus formularios.



Todas las colecciones tienen _id (ObjectId) generado automáticamente por MongoDB. "Condicional" indica que el campo es obligatorio solo en el caso descrito en las reglas.

Colaborador

Campo	Tipo	Obligatorio	Único	Reglas de validación
nombre	String	Sí	No	Sin espacios al inicio o al final; de 3 a 60 caracteres; solo letras (con tildes y ñ) y espacios
correo	String	Sí	Sí	Se guarda en minúsculas; formato de correo válido; si el rol es administrador_general, debe terminar en @mep.go.cr
rol	String	Sí	No	enum: administrador_general, tecnico, director
contrasenaHash	String	Condicional: tecnico y director	No	Se guarda el hash de bcrypt, nunca la contraseña; la contraseña recibida debe tener mínimo 8 caracteres, al menos una letra y un

				número; no se incluye en las respuestas de la API
zonas	[String]	Condicional: tecnico	No	enum: las 7 provincias; de 1 a 7 elementos, sin repetidos; vacío para los otros roles
centrosAsignados	[ObjectId → CentroEducativo]	Condicional: director	No	De 1 a 3 elementos; cada id debe existir en CentroEducativo; vacío para los otros roles
fechaRegistro	Date	Sí	No	Por defecto, la fecha y hora del registro

CentroEducativo

Campo	Tipo	Obligatorio	Único	Reglas de validación
nombre	String	Sí	No	Sin espacios al inicio o al final; de 3 a 150 caracteres

codigoPresupuestario	String	Sí	Sí	Solo dígitos
tipoCentro	String	Sí	No	enum: preescolar, diurno, nocturno
ofertaEducativa	[String]	Sí	No	Al menos 1 elemento; enum: Preescolar, I y II Ciclo, III Ciclo y Diversificada, Educación de adultos
cantidadEstudiantes	Number	Sí	No	Número entero, mínimo 0
direccion	Objeto embebido	Sí	No	provincia (enum: 7 provincias), canton y distrito obligatorios; senas opcional, máximo 250 caracteres
telefono	String	No	No	8 dígitos, con guion opcional (2222-2222)
correo	String	No	No	Formato de correo válido

Formulario

Campo	Tipo	Obligatorio	Único	Reglas de validación
nombre	String	Sí	No	De 5 a 100 caracteres
descripcion	String	Sí	No	De 15 a 500 caracteres
preguntas	[Subdocumento]	Sí	No	Al menos 1 pregunta; cada pregunta tiene su propio _id
preguntas.texto	String	Sí	No	Mínimo 5 caracteres
preguntas.tipoRespuesta	String	Sí	No	enum: unica, multiple, abierta
preguntas.opciones	[String]	Condicional: unica y multiple	No	Mínimo 2 opciones, sin repetidas; vacío si el tipo es abierta
preguntas.obligatoria	Boolean	Sí	No	Por defecto true

ofertasAplicables	[String]	Sí	No	Al menos 1 elemento; mismo enum que ofertaEducativa
estado	String	Sí	No	enum: borrador, publicado; por defecto borrador
creadoPor	ObjectId → Colaborador	Sí	No	Debe ser un colaborador con rol administrador_general
fechaCreacion	Date	Sí	No	Por defecto, la fecha y hora de creación

Censo

Campo	Tipo	Obligatorio	Único	Reglas de validación
nombre	String	Sí	No	De 5 a 120 caracteres

fechaApertura	Date	Sí	No	Al crear el censo, no puede ser anterior a la fecha actual
fechaCierre	Date	Sí	No	Posterior a fechaApertura
estado	String	Sí	No	enum: borrador, abierto, pausado, cerrado; por defecto borrador
historialEstados	[Subdocumento]	No	No	Se agrega un elemento en cada cambio de estado: estado (mismo enum) y fecha (Date, por defecto la fecha y hora del cambio)
formularios	[ObjectId → Formulario]	Sí	No	De 1 a 14 elementos; solo formularios en estado publicado

Los centros asignados a un censo no se guardan en esta colección: se obtienen de los documentos de RespuestaCenso que se crean al asignarlo.

RespuestaCenso

Campo	Tipo	Obligatorio	Único	Reglas de validación
censo	ObjectId → Censo	Sí	Compuesto *	Debe existir
formulario	ObjectId → Formulario	Sí	Compuesto *	Debe estar incluido en censo.formularios
centroEducativo	ObjectId → CentroEducativo	Sí	Compuesto *	Su ofertaEducativa debe coincidir con al menos una de las ofertasAplicables del formulario
respuestas	[Subdocumento]	No	No	Vacío al asignar el censo. preguntaId (ObjectId de una pregunta del formulario) y valor ([String]): 1 elemento si la pregunta es abierta o unica, 1 o más si es multiple; en las preguntas de selección, los valores

				deben ser opciones de la pregunta
estado	String	Sí	No	enum: borrador, en_registro, en_revision, en_subsane, aceptado; por defecto borrador
motivoSubsane	String	Condicional: en_subsane	No	Mínimo 20 caracteres
completadoPor	ObjectId → Colaborador	Condicional: al enviar	No	Debe ser un colaborador con rol director
revisadoPor	ObjectId → Colaborador	Condicional: al aceptar o devolver	No	Debe ser un colaborador con rol tecnico
fechaEnvio	Date	Condicional: al enviar	No	Fecha y hora del envío

fechaRevision	Date	Condicional: al aceptar o devolver	No	Posterior a fechaEnvio

* Índice único compuesto por censo + formulario + centroEducativo: un centro tiene un solo documento por cada formulario de un censo.

Relaciones y cardinalidad

Las colecciones se relacionan con 8 referencias (ObjectId) y 4 subdocumentos embebidos. Se referencia cuando la otra entidad existe por sí sola o puede crecer sin límite; se embebe cuando los datos solo tienen sentido dentro del documento y son pocos.

Referencias

Relación	**Origen → destino (campo)**	**Cardinalidad**	**Justificación**
crear	Formulario → Colaborador (creadoPor)	N : 1	Un administrador crea muchos formularios; cada formulario tiene un solo creador
administrar	Colaborador → CentroEducativo (centrosAsignados)	N : N	Un director administra de 1 a 3 centros y un centro puede tener más de un director o encargado; el

			arreglo está acotado a 3, por eso se guarda del lado del director
incluir	Censo → Formulario (formularios)	N : N	Un censo incluye hasta 14 formularios y un formulario se reutiliza en varios censos
pertenecer	RespuestaCenso → Censo (censo)	N : 1	Un censo genera un documento por cada centro y formulario (cientos o miles); embeberlos haría crecer el censo sin límite
responder	RespuestaCenso → Formulario (formulario)	N : 1	Cada documento responde un solo formulario; el formulario se mantiene independiente
remitir	RespuestaCenso → CentroEducativo (centroEducativo)	N : 1	Un centro remite un documento por cada formulario de cada censo
completar	RespuestaCenso → Colaborador (completadoPor)	N : 1	El director que envía el formulario; un director completa muchos

revisar	RespuestaCenso → Colaborador (revisadoPor)	N : 1	El técnico que acepta o devuelve el formulario; un técnico revisa muchos

La relación entre Censo y CentroEducativo es N : N y se representa mediante RespuestaCenso: asignar un censo a un centro crea un documento por cada formulario aplicable a su oferta educativa.

Subdocumentos embebidos

Colección	**Campo**	**Cardinalidad**	**Justificación**
CentroEducativo	direccion	1 : 1	La dirección solo pertenece a ese centro y siempre se muestra con sus datos precargados (RF-015)
Formulario	preguntas	1 : pocos	Se crean, editan y leen junto con el formulario (RF-001 a RF-003)
Censo	historialEstados	1 : pocos	Cada cambio manual de estado (RF-006) agrega un elemento; son pocos cambios por censo

RespuestaCenso	respuestas	1 : pocos	Una respuesta por pregunta del formulario; se guardan y envían juntas (RF-009)

Estructura del sistema

El código del sistema vive en la rama feature del repositorio, separado en dos carpetas: front-end/ (la vista) y back-end/ (la API, con los controladores y modelos). La documentación se mantiene en la rama docs.

Rama	Contenido
main	Rama principal; solo recibirá el código de feature mediante un Pull Request cuando esté lista la versión final del proyecto; hasta entonces no recibe commits
feature	Código del sistema: front-end/ y back-end/
docs	Documentación del proyecto (ERS, Especificación de Diseño, matriz de trazabilidad, diagramas)

SOFT-11-C1-Proyecto-Integrador-1/ (rama feature)

├── front-end/

│ ├── index.html, login.html, login-admin.html, dashboard-admin.html,

│ │ formulario-registrar.html, censo-listar.html, colaborador-listar.html,

```
│   │   dashboard-director.html, dashboard-tecnico.html
│   ├── css/
│   └── js/
├── back-end/
│   ├── src/
│   │   ├── auth/
│   │   ├── colaboradores/
│   │   ├── centros-educativos/
│   │   ├── formularios/            (misma estructura en cada recurso)
│   │   │   ├── dto/
│   │   │   │   ├── crear-formulario.dto.ts
│   │   │   │   └── actualizar-formulario.dto.ts
│   │   │   ├── schemas/
│   │   │   │   └── formulario.schema.ts
│   │   │   ├── formularios.controller.ts
│   │   │   ├── formularios.module.ts
│   │   │   └── formularios.service.ts
│   │   ├── censos/
│   │   ├── respuestas-censo/
│   │   ├── common/
│   │   ├── seeders/
│   │   ├── app.module.ts
│   │   └── main.ts
│   ├── .env.example
│   ├── .gitignore
│   ├── nest-cli.json
│   ├── package.json
│   ├── package-lock.json
```

```
│   ├── pruebas.http
│   ├── README.md
│   ├── tsconfig.json
│   └── tsconfig.build.json
└── README.md
```

Front-end

Carpeta / archivo	**Descripción**
front-end/ (raíz)	Carpeta raíz del front-end: contiene las 9 pantallas HTML y las carpetas css/ y js/. index.html es la selección de usuario y la página de entrada
css/	Estilos propios: estilo.css (variables de color, modo oscuro y componentes comunes), login.css (pantallas de acceso), navbar.css (barra lateral y encabezado) y dashboard.css (contenido de los paneles)
js/	Scripts: tema.js (modo oscuro), validaciones.js (funciones de validación y mensajes compartidas) y un archivo por pantalla con su lógica de interfaz y sus llamadas a la API

En el prototipo actual, js/datos-ejemplo.js contiene datos simulados; se reemplazará por las llamadas a la API con fetch.

Back-end

Carpeta / archivo	Descripción
back-end/ (raíz)	Carpeta raíz del back-end, creada con nest new (gestor npm, módulos CJS)
src/	Código fuente en TypeScript
src/<recurso>/	Un módulo por recurso, creado con nest g resource (REST API): auth, colaboradores, centros-educativos, formularios, censos y respuestas-censo. Cada uno contiene los archivos de las filas siguientes
<recurso>.controller.ts	**Controlador (MVC).** Define las rutas del recurso, aplica los guards y responde en JSON
<recurso>.service.ts	**Modelo (MVC), lógica.** Reglas de negocio y consultas a la base de datos con Mongoose
<recurso>.module.ts	Registra el controlador, el servicio y el schema del recurso
dto/	Clases que describen y validan con class-validator los datos recibidos, tanto en el cuerpo de la petición como en

	los filtros de la URL (?estado=, ?centro=), por ejemplo crear-censo.dto.ts y actualizar-censo.dto.ts
schemas/	**Modelo (MVC), datos.** Schema de Mongoose de la colección con sus tipos y validaciones, por ejemplo censo.schema.ts. El módulo auth no tiene schema propio: usa el de colaboradores
src/common/	Elementos compartidos entre módulos: guards (token JWT y rol), el decorador @Roles y funciones auxiliares, como calcular los días restantes de un censo
src/seeders/	Script que carga los datos iniciales: los centros educativos precargados (RF-015) y el primer administrador general
src/app.module.ts	Módulo principal: importa los módulos de los recursos y configura la conexión con MongoDB Atlas y las variables de entorno
src/main.ts	Punto de entrada: crea la aplicación, activa CORS, aplica ValidationPipe de forma global con las opciones whitelist (descarta los campos que no están en el DTO), forbidNonWhitelisted (responde 400 si se envían campos

	de más) y transform (convierte los datos a los tipos del DTO), y levanta el servidor
package.json y package-lock.json	Dependencias y comandos (npm run start:dev, npm run build)
nest-cli.json, tsconfig.json y tsconfig.build.json	Configuración del CLI de NestJS y del compilador de TypeScript
pruebas.http	Peticiones de prueba a los endpoints, ejecutables desde VS Code
README.md	Instrucciones para instalar y ejecutar el back-end
.env.example	Plantilla de las variables de entorno (PORT, MONGODB_URI, JWT_SECRET, JWT_EXPIRES_IN); el archivo .env real no se sube al repositorio
.gitignore	Excluye node_modules/, dist/ (código compilado) y .env del repositorio

API REST

La API tiene 34 endpoints agrupados en 6 recursos; cada ruta empieza con el nombre del recurso, como en los proyectos vistos en clase. Todos reciben y responden JSON; salvo los dos de inicio de sesión, todos requieren el token JWT y verifican el rol indicado.

Autenticación

RF	Método	Ruta	Descripción	Rol
RF-016	POST	/auth/login	Inicia la sesión de técnicos y directores con correo y contraseña; devuelve el token JWT y el rol	Público
RF-016	POST	/auth/mep	Inicia la sesión de la Jefatura: valida que el correo sea @mep.go.cr y pertenezca a un administrador general, autentica con el sistema del MEP (simulado en el proyecto) y devuelve el token JWT	Público
RF-016	GET	/auth/perfil	Devuelve los datos del usuario autenticado: nombre, rol y zonas o centros asignados	Todos

Colaboradores

RF	Método	Ruta	Descripción	Rol
RF-004	GET	/colaboradores	Lista los colaboradores; filtro opcional ?rol=	Administrador
RF-004	GET	/colaboradores/:id	Devuelve un colaborador	Administrador
RF-004, RF-008	POST	/colaboradores	Registra un colaborador con su rol, sus zonas (técnico) o sus centros (director) y una contraseña temporal	Administrador
RF-004, RF-008	PUT	/colaboradores/:id	Actualiza los datos, el rol, las zonas o los centros de un colaborador	Administrador
RF-004	DELETE	/colaboradores/:id	Elimina un colaborador	Administrador

Centros educativos

RF	Método	Ruta	Descripción	Rol
RF-004, RF-005	GET	/centros-educativos	Lista los centros para asignarlos a directores y censos; filtros opcionales ?provincia= y ?oferta=	Administrador
RF-008, RF-014	GET	/centros-educativos/asignados	Devuelve los centros del usuario: los de las zonas del técnico o los asignados al director	Técnico, Director
RF-015	GET	/centros-educativos/:id	Devuelve los datos precargados de un centro	Administrador, Técnico y Director (solo sus centros)

Formularios

RF	Método	Ruta	Descripción	Rol
RF-001, RF-005	GET	/formularios	Lista los formularios; filtros opcionales ?estado= y ?oferta=	Administrador
RF-002	GET	/formularios/:id	Devuelve un formulario con sus preguntas	Administrador
RF-001	POST	/formularios	Crea un formulario desde cero	Administrador
RF-001	POST	/formularios/:id/copiar	Crea un formulario a partir de uno existente (plantilla), copiando sus preguntas en estado borrador	Administrador

RF-002	PUT	/formularios/:id	Actualiza el nombre, la descripción, las ofertas aplicables o el estado	Administrador
RF-002	DELETE	/formularios/:id	Elimina un formulario; responde 409 si está incluido en un censo	Administrador
RF-003	POST	/formularios/:id/preguntas	Agrega una pregunta con su tipo de respuesta y sus opciones	Administrador
RF-003	PUT	/formularios/:id/preguntas/:preguntaId	Edita el texto, el tipo, las opciones o la obligatoriedad de una pregunta	Administrador
RF-003	DELETE	/formularios/:id/preguntas/:preguntaId	Elimina una pregunta	Administrador

Censos

RF	Método	Ruta	Descripción	Rol
RF-005	GET	/censos	Lista los censos; filtro opcional ?estado=	Administrador
RF-007	GET	/censos/metricas	Devuelve las métricas del dashboard: censos activos, formularios creados, centros con censo activo, respuestas recibidas y días restantes de cada censo	Administrador
RF-014	GET	/censos/disponibles	Devuelve los censos abiertos con los formularios que aplican a la oferta de cada centro del director; filtro opcional ?centro=	Director
RF-005	GET	/censos/:id	Devuelve un censo con sus formularios y sus centros asignados	Administrador

RF-005	POST	/censos	Crea un censo con nombre, fechas, formularios y centros; genera un documento de RespuestaCenso por cada centro y formulario aplicable	Administrador
RF-005	PUT	/censos/:id	Actualiza la configuración de un censo en estado borrador o pausado	Administrador
RF-006, RF-012	PATCH	/censos/:id/estado	Abre, pausa, reanuda o cierra un censo y registra el cambio en historialEstados	Administrador
RF-005	DELETE	/censos/:id	Elimina un censo en estado borrador y sus documentos de RespuestaCenso	Administrador

Respuestas de censo

RF	Método	Ruta	Descripción	Rol
RF-008, RF-013	GET	/respuestas-censo	Lista los formularios asignados con su estado; filtros opcionales ?censo=, ?centro= y ?estado=. El técnico solo ve los de su zona y el director solo los de sus centros	Administrador, Técnico, Director
RF-009, RF-015	GET	/respuestas-censo/:id	Devuelve las preguntas del formulario, las respuestas guardadas, el estado y los datos precargados del centro	Técnico, Director
RF-009, RF-012	PUT	/respuestas-censo/:id	Guarda el avance de las respuestas sin enviarlas	Director
RF-009, RF-012	PATCH	/respuestas-censo/:id /enviar	Remite el formulario al técnico: verifica que el censo esté abierto y que	Director

			las preguntas obligatorias tengan respuesta	
RF-010, RF-012	PATCH	/respuestas-censo/:id /devolver	Devuelve el formulario al director con el motivo de subsane	Técnico
RF-011, RF-012	PATCH	/respuestas-censo/:id /aceptar	Acepta el formulario	Técnico

Cambios de estado (RF-012)

Colección	**Estado actual**	**Endpoint**	**Nuevo estado**	**Rol**
RespuestaCenso	—	POST /censos (al asignar el censo)	borrador	Administrador
RespuestaCenso	borrador	PUT /respuestas-censo/:id	en_registro	Director
RespuestaCenso	en_registro o en_subsane	PATCH /respuestas-censo/:id/ enviar	en_revision	Director

RespuestaCenso	en_revision	PATCH /respuestas-censo/:id/devolver	en_subsane	Técnico
RespuestaCenso	en_revision	PATCH /respuestas-censo/:id/aceptar	aceptado	Técnico
Censo	borrador o pausado	PATCH /censos/:id/estado	abierto	Administrador
Censo	abierto	PATCH /censos/:id/estado	pausado	Administrador
Censo	abierto o pausado	PATCH /censos/:id/estado	cerrado	Administrador

Un cambio que no esté en esta tabla se rechaza con 409. Guardar avance en en_subsane no cambia el estado.

Códigos de respuesta

Código	Significado en SICEN
200	Consulta o actualización correcta
201	Recurso creado
400	Datos con formato incorrecto (DTO y ValidationPipe)
401	Falta el token o no es válido
403	El rol no tiene permiso para esa acción
404	El recurso no existe
409	Conflicto con una regla de negocio: correo repetido, censo cerrado, cambio de estado no permitido

Tecnologías utilizadas

El front-end usa HTML, CSS y JavaScript con Bootstrap 5.3.6; el back-end usa NestJS 12 con TypeScript 6 sobre Node.js 24, y Mongoose 9 con MongoDB Atlas.

Front-end

Tecnología	Versión	Propósito
HTML	5	Estructura de las pantallas
CSS	3	Diseño visual, paleta del MEP y modo oscuro
JavaScript	ES6 (ECMAScript 2015) o superior	Interactividad, validación de formularios y comunicación con la API mediante fetch
Bootstrap	5.3.6	Framework CSS: cuadrícula adaptable, componentes (tablas, formularios, menú deslizable) y modo oscuro con data-bs-theme
SweetAlert2	11	Ventanas de confirmación, error y éxito
Font Awesome Free	6.5.1	Íconos de la interfaz

Libre Baskerville (Google Fonts)	Pesos 400 y 700	Tipografía institucional, adaptación web de Baskerville según el manual de identidad del MEP

Back-end y base de datos

Tecnología	Versión	Propósito
Node.js	24.21.0 (LTS)	Entorno de ejecución de JavaScript en el servidor
npm	12.2.0	Gestor de paquetes y comandos del proyecto
TypeScript	6.0.3	Lenguaje del back-end: JavaScript con tipos, compilado a JavaScript
NestJS (@nestjs/core, @nestjs/common)	12.4.2	Framework del back-end: módulos, controladores, servicios, guards y pipes de la API REST
@nestjs/platform-express	12.4.2	Servidor HTTP que NestJS usa internamente (basado en Express)

Nest CLI (@nestjs/cli)	12.0.10	Crear el proyecto (nest new) y los recursos (nest g resource)
@nestjs/schematics	12.0.13	Plantillas que usa el CLI para generar módulos, controladores y servicios
reflect-metadata	0.2.2	Permite que NestJS lea los decoradores (@Controller, @Prop, @IsString, etc.)
rxjs	7.8.2	Biblioteca de programación reactiva que NestJS necesita internamente
Mongoose	9.12.1	Modelado de datos: schemas, validaciones y consultas a MongoDB
@nestjs/mongoose	12.0.1	Integración de Mongoose con NestJS: schemas con decoradores e @InjectModel
MongoDB Atlas	Servicio en la nube (plan gratuito)	Base de datos NoSQL donde se guardan las 5 colecciones

@nestjs/config	12.1.1	Cargar las variables de entorno del archivo .env (cadena de conexión, clave del JWT)
@nestjs/jwt	12.0.4	Generar y verificar los tokens JWT de sesión
bcrypt	6.0.0	Cifrar las contraseñas de técnicos y directores antes de guardarlas
class-validator	0.15.1	Reglas de validación de los DTO
class-transformer	0.5.1	Convertir el JSON recibido en objetos de los DTO para validarlos
@nestjs/mapped-types	12.0.1	Crear los DTO de actualización a partir de los de creación (PartialType)

Herramientas

Herramienta	**Versión**	**Propósito**
Git	2.56.0	Control de versiones

GitHub	Servicio web	Repositorio remoto y ramas de trabajo (main, docs, feature)
Jira	Servicio web	Gestión de historias de usuario y sprints
Figma	Servicio web	Wireframes de media fidelidad
Visual Studio Code	1.145.2	Editor de código
WAVE	Servicio web	Verificación de accesibilidad (RNF-002)

Node.js, npm, el Nest CLI, Git y VS Code son las versiones instaladas en el equipo de desarrollo (node -v, npm -v, nest -v, git --version, code --version). Los paquetes del back-end son los que instalará un proyecto creado con nest new y el Nest CLI 12, en su versión publicada en npm al 10 de noviembre de 2026; al crear el proyecto se confirman con npm ls --depth=0 dentro de la carpeta back-end.